

```

# =====

# SCENARIO 1: FLOOD-RELIEF DRONE ROUTING

# Greedy Best-First Search vs A*

# =====


class FloodGrid:
    """
    A grid-based representation of a flood-affected region.
    Each cell has a movement cost (terrain/weather), and some cells can
    become blocked dynamically (simulating real-time flood updates).
    """

    def __init__(self, width: int, height: int, seed: int = 42):
        self.width = width
        self.height = height
        random.seed(seed)

        # Movement cost per cell: 1 = normal, higher = harsher weather/terrain
        self.cost = {
            (x, y): random.choice([1, 1, 1, 2, 3])
            for x in range(width) for y in range(height)
        }

        self.blocked: Set[Tuple[int, int]] = set()

    def block_random_cells(self, n: int):
        """Simulate sudden flood damage blocking n random cells (dynamic environment)."""
        candidates = [c for c in self.cost if c not in self.blocked]
        for cell in random.sample(candidates, min(n, len(candidates))):
            self.blocked.add(cell)

    def neighbors(self, node: Tuple[int, int]):
        x, y = node

```

```

for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
    nx, ny = x + dx, y + dy
    if 0 <= nx < self.width and 0 <= ny < self.height:
        if (nx, ny) not in self.blocked:
            yield (nx, ny)

```

```

def step_cost(self, node: Tuple[int, int]) -> float:
    return self.cost[node]

```

```

def straight_line_heuristic(a: Tuple[int, int], b: Tuple[int, int]) -> float:
    """Admissible heuristic: Euclidean straight-line distance."""
    return math.hypot(a[0] - b[0], a[1] - b[1])

```

```

def reconstruct_path(came_from: Dict, current) -> List:
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path

```

```

def greedy_best_first_search(grid: FloodGrid, start, goal, dynamic_updates=False):
    """
    Greedy Best-First Search: expands the node that appears closest to the
    goal according to the heuristic h(n) ONLY -- ignores accumulated cost g(n).
    Fast, but not guaranteed optimal or even complete under obstacles.
    """
    frontier = [(straight_line_heuristic(start, goal), start)]

```

```

came_from = {}
visited = {start}
nodes_expanded = 0

while frontier:
    _, current = heapq.heappop(frontier)
    nodes_expanded += 1

    if dynamic_updates and nodes_expanded == 15:
        # Simulate a real-time update: new blockage appears mid-search
        grid.block_random_cells(3)

    if current == goal:
        return reconstruct_path(came_from, current), nodes_expanded

    for nxt in grid.neighbors(current):
        if nxt not in visited:
            visited.add(nxt)
            came_from[nxt] = current
            priority = straight_line_heuristic(nxt, goal) # h(n) only
            heapq.heappush(frontier, (priority, nxt))

    return None, nodes_expanded # No path found (can happen if greedily boxed in)

```

```

def a_star_search(grid: FloodGrid, start, goal, dynamic_updates=False):

```

```

    """

```

A* Search: expands nodes minimizing $f(n) = g(n) + h(n)$, where $g(n)$ is the real accumulated movement cost and $h(n)$ is the straight-line estimate.

Guaranteed optimal if $h(n)$ is admissible (never overestimates).

```

    """

```

```

frontier = [(straight_line_heuristic(start, goal), start)]
came_from = {}
g_score = {start: 0}
nodes_expanded = 0

while frontier:
    _, current = heapq.heappop(frontier)
    nodes_expanded += 1

    if dynamic_updates and nodes_expanded == 15:
        grid.block_random_cells(3) # same real-time disruption as above

    if current == goal:
        return reconstruct_path(came_from, current), g_score[current], nodes_expanded

    for nxt in grid.neighbors(current):
        tentative_g = g_score[current] + grid.step_cost(nxt)
        if nxt not in g_score or tentative_g < g_score[nxt]:
            g_score[nxt] = tentative_g
            came_from[nxt] = current
            f_score = tentative_g + straight_line_heuristic(nxt, goal)
            heapq.heappush(frontier, (f_score, nxt))

    return None, float("inf"), nodes_expanded

def path_true_cost(grid: FloodGrid, path: List[Tuple[int, int]]) -> float:
    if not path:
        return float("inf")
    return sum(grid.cost[c] for c in path[1:])

```

```

def demo_scenario_1():
    print("=" * 70)
    print("SCENARIO 1: Flood-Relief Drone Routing")
    print("=" * 70)
    grid = FloodGrid(15, 15)
    start, goal = (0, 0), (14, 14)

    g_path, g_nodes = greedy_best_first_search(FloodGrid(15, 15), start, goal)
    a_path, a_cost, a_nodes = a_star_search(FloodGrid(15, 15), start, goal)

    fresh_grid_for_cost = FloodGrid(15, 15)
    print(f"Greedy Best-First -> nodes expanded: {g_nodes}, "
          f"path length: {len(g_path) if g_path else 'NO PATH'}, "
          f"true cost: {path_true_cost(fresh_grid_for_cost, g_path):.1f}")
    print(f"A* Search      -> nodes expanded: {a_nodes}, "
          f"path length: {len(a_path) if a_path else 'NO PATH'}, "
          f"true cost: {a_cost:.1f}")
    print("\n--- With dynamic blockages appearing mid-search ---")
    g_path2, g_nodes2 = greedy_best_first_search(FloodGrid(15, 15), start, goal,
dynamic_updates=True)

    a_path2, a_cost2, a_nodes2 = a_star_search(FloodGrid(15, 15), start, goal,
dynamic_updates=True)

    print(f"Greedy (dynamic) -> nodes expanded: {g_nodes2}, "
          f"path found: {g_path2 is not None}")
    print(f"A* (dynamic)   -> nodes expanded: {a_nodes2}, "
          f"path found: {a_path2 is not None}, cost: {a_cost2:.1f}")
    print()

# =====
# SCENARIO 2: SMART-CITY TRAFFIC SIGNAL OPTIMIZATION
# Hill Climbing vs Simulated Annealing (local search / metaheuristics)
# =====

```

```
class TrafficNetwork:
```

```
    """
```

```
    Simplified model: N intersections, each with a signal-timing setting
    (green-light duration in seconds, 10-60s). The objective function
    approximates total waiting time + fuel + congestion as a nonlinear,
    multi-modal cost surface (to simulate local optima in real traffic systems).
```

```
    """
```

```
    def __init__(self, n_intersections: int, seed: int = 7):
```

```
        self.n = n_intersections
```

```
        random.seed(seed)
```

```
        # Random "ideal" timing per intersection + coupling between neighbors
```

```
        self.ideal = [random.randint(20, 45) for _ in range(n_intersections)]
```

```
        self.coupling = [random.uniform(0.1, 0.4) for _ in range(n_intersections)]
```

```
    def cost(self, timings: List[int]) -> float:
```

```
        """
```

```
        Lower is better. Multi-modal: combines a smooth quadratic term with a
        oscillating term to create local minima/plateaus/ridges, just like
        real traffic optimization landscapes.
```

```
        """
```

```
        total = 0.0
```

```
        for i, t in enumerate(timings):
```

```
            diff = t - self.ideal[i]
```

```
            smooth = 0.5 * diff ** 2
```

```
            oscillation = 40 * math.sin(t / 2.2) * self.coupling[i]
```

```
            neighbor_penalty = 0
```

```
            if i > 0:
```

```
                neighbor_penalty = 0.2 * abs(timings[i] - timings[i - 1])
```

```
            total += smooth + oscillation + neighbor_penalty
```

```
    return total
```

```
def random_state(self) -> List[int]:
```

```
    return [random.randint(10, 60) for _ in range(self.n)]
```

```
def hill_climbing(network: TrafficNetwork, max_iterations=500):
```

```
    """
```

Steepest-ascent (descent, since minimizing) hill climbing.

Only accepts moves that strictly improve cost -> gets stuck at the first local minimum, plateau, or ridge it encounters.

```
    """
```

```
    current = network.random_state()
```

```
    current_cost = network.cost(current)
```

```
    history = [current_cost]
```

```
    for _ in range(max_iterations):
```

```
        best_neighbor, best_cost = None, current_cost
```

```
        for i in range(network.n):
```

```
            for delta in (-5, -1, 1, 5):
```

```
                neighbor = current.copy()
```

```
                neighbor[i] = max(10, min(60, neighbor[i] + delta))
```

```
                c = network.cost(neighbor)
```

```
                if c < best_cost:
```

```
                    best_cost, best_neighbor = c, neighbor
```

```
        if best_neighbor is None:
```

```
            break # local optimum / plateau reached: no improving neighbor
```

```
        current, current_cost = best_neighbor, best_cost
```

```
        history.append(current_cost)
```

```
    return current, current_cost, history
```

```

def simulated_annealing(network: TrafficNetwork, max_iterations=2000,
                        initial_temp=50.0, cooling_rate=0.995):
    """
    Simulated Annealing: like hill climbing, but occasionally accepts WORSE
    moves (probability shrinks as "temperature" cools). This lets it escape
    local minima, plateaus, and ridges that trap plain hill climbing.
    """
    current = network.random_state()
    current_cost = network.cost(current)
    best, best_cost = current.copy(), current_cost
    temp = initial_temp
    history = [current_cost]

    for _ in range(max_iterations):
        i = random.randrange(network.n)
        delta = random.choice([-5, -3, -1, 1, 3, 5])
        neighbor = current.copy()
        neighbor[i] = max(10, min(60, neighbor[i] + delta))
        neighbor_cost = network.cost(neighbor)

        cost_diff = neighbor_cost - current_cost
        if cost_diff < 0 or random.random() < math.exp(-cost_diff / max(temp, 1e-6)):
            current, current_cost = neighbor, neighbor_cost
            if current_cost < best_cost:
                best, best_cost = current.copy(), current_cost

        temp *= cooling_rate
        history.append(current_cost)

```

```
return best, best_cost, history
```

```
def demo_scenario_2():
```

```
    print("=" * 70)
```

```
    print("SCENARIO 2: Smart-City Traffic Signal Optimization")
```

```
    print("=" * 70)
```

```
    network = TrafficNetwork(n_intersections=12)
```

```
    # Run each algorithm from several random restarts and average, since a
```

```
    # single run can get lucky/unlucky with its starting point.
```

```
    hc_results = []
```

```
    for run in range(8):
```

```
        random.seed(100 + run)
```

```
        hc_results.append(hill_climbing(network)[1])
```

```
    sa_results = []
```

```
    for run in range(8):
```

```
        random.seed(100 + run)
```

```
        sa_results.append(simulated_annealing(network)[1])
```

```
    avg_hc, best_hc = sum(hc_results) / len(hc_results), min(hc_results)
```

```
    avg_sa, best_sa = sum(sa_results) / len(sa_results), min(sa_results)
```

```
    print(f"Hill Climbing    -> avg cost over 8 runs: {avg_hc:.2f}, best: {best_hc:.2f} "
```

```
          f"(frequently stuck at local optima)")
```

```
    print(f"Simulated Annealing -> avg cost over 8 runs: {avg_sa:.2f}, best: {best_sa:.2f} "
```

```
          f"(consistently escapes local optima)")
```

```
    improvement = (avg_hc - avg_sa) / avg_hc * 100 if avg_hc else 0
```

```
    print(f"SA improved average cost over Hill Climbing by ~{improvement:.1f}%")
```

```
    print()
```

```
# =====  
# SCENARIO 3: MARS ROVER - ONLINE SEARCH AGENT  
# LRTA*-style online search with partial observability  
# =====
```

```
class MarsTerrain:
```

```
    """
```

```
The TRUE terrain (hazards) is hidden from the rover. The rover only  
discovers hazards within its local sensor range as it moves --  
a classic partially-observable, dynamic search problem.
```

```
    """
```

```
def __init__(self, width, height, hazard_density=0.2, seed=99):
```

```
    random.seed(seed)
```

```
    self.width, self.height = width, height
```

```
    self.hazards = {
```

```
        (x, y) for x in range(width) for y in range(height)
```

```
        if random.random() < hazard_density
```

```
    }
```

```
def is_hazard(self, cell):
```

```
    return cell in self.hazards
```

```
def in_bounds(self, cell):
```

```
    x, y = cell
```

```
    return 0 <= x < self.width and 0 <= y < self.height
```

```
class MarsRoverAgent:
```

"""

Online search agent (LRTA*-like). Unlike offline A*, the rover does NOT know the full map in advance. It senses its immediate surroundings, updates an internal cost estimate table, and decides its next move one step at a time -- committing to real physical movement each turn.

"""

SENSOR_RANGE = 1 # can only sense adjacent cells before moving

def __init__(self, terrain: MarsTerrain, start, goal):

self.terrain = terrain

self.pos = start

self.goal = goal

self.H: Dict = {} # learned heuristic estimates, updated online

self.known_hazards: Set = set()

self.visited_path = [start]

def h(self, cell):

if cell not in self.H:

self.H[cell] = straight_line_heuristic(cell, self.goal)

return self.H[cell]

def sense(self):

"""Discover hazards in sensor range around the current position (partial observability)."""

x, y = self.pos

for dx in range(-self.SENSOR_RANGE, self.SENSOR_RANGE + 1):

for dy in range(-self.SENSOR_RANGE, self.SENSOR_RANGE + 1):

cell = (x + dx, y + dy)

if self.terrain.in_bounds(cell) and self.terrain.is_hazard(cell):

self.known_hazards.add(cell)

```

def neighbors(self, cell):
    x, y = cell
    for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nxt = (x + dx, y + dy)
        if self.terrain.in_bounds(nxt) and nxt not in self.known_hazards:
            yield nxt

def step(self):
    """One iteration of LRTA*: sense, evaluate neighbors, move, update model."""
    self.sense()
    neighbors = list(self.neighbors(self.pos))
    if not neighbors:
        return False # stuck -- would trigger a re-planning/backtrack in practice

    # Estimate cost of each neighbor as step cost (1) + learned heuristic
    costs = {n: 1 + self.h(n) for n in neighbors}
    best_neighbor = min(costs, key=costs.get)

    # LRTA* update rule: improve current cell's heuristic estimate
    self.H[self.pos] = max(self.h(self.pos), costs[best_neighbor])

    self.pos = best_neighbor
    self.visited_path.append(self.pos)
    return True

def run(self, max_steps=300):
    steps = 0
    while self.pos != self.goal and steps < max_steps:
        moved = self.step()
        if not moved:
            break

```

```

        steps += 1

    return self.pos == self.goal, steps

def demo_scenario_3():
    print("=" * 70)
    print("SCENARIO 3: Mars Rover Online Search (Partial Observability)")
    print("=" * 70)
    terrain = MarsTerrain(20, 20, hazard_density=0.15)
    rover = MarsRoverAgent(terrain, start=(0, 0), goal=(19, 19))
    reached, steps = rover.run()

    print(f'Rover reached goal: {reached}')
    print(f'Steps taken (online, no full map known in advance): {steps}')
    print(f'Total hazards in world: {len(terrain.hazards)}, "
          f'hazards discovered via sensing: {len(rover.known_hazards)}')
    print("Note: an OFFLINE search (e.g. plain A*) would require the full "
          "map upfront, which is unrealistic for real Mars exploration.")
    print()

# =====
# SCENARIO 4: UNIVERSITY EXAM TIMETABLING
# CSP with Backtracking Search + Forward Checking
# =====

@dataclass
class ExamCSP:
    """
    Variables : exams (one per course)

    Domains : available (hall, timeslot) pairs

```

Constraints:

- No student takes two exams in the same slot (overlap constraint)
- Limited halls per slot (capacity constraint)
- Some subjects must precede others (precedence constraint)
- Faculty availability (per-exam allowed slots)

"""

exams: List[str]

halls: List[str]

timeslots: List[str]

student_exams: Dict[str, List[str]] # student -> list of exam names

precedence: List[Tuple[str, str]] # (before, after) exam name pairs

faculty_unavailable: Dict[str, Set[str]] = field(default_factory=dict) # exam -> forbidden slots

def domain(self, exam) -> List[Tuple[str, str]]:

forbidden = self.faculty_unavailable.get(exam, set())

return [(h, t) for h in self.halls for t in self.timeslots if t not in forbidden]

def exams_of_same_student(self, exam) -> List[str]:

related = set()

for exams in self.student_exams.values():

if exam in exams:

related.update(e for e in exams if e != exam)

return list(related)

def consistent(self, exam, value, assignment: Dict[str, Tuple[str, str]]) -> bool:

hall, slot = value

Capacity / same-hall-same-slot constraint

for other_exam, other_value in assignment.items():

if other_value == (hall, slot):

return False # two exams can't share the same hall & slot

```
# No overlapping exams for the same student
```

```
for other_exam in self.exams_of_same_student(exam):
```

```
    if other_exam in assignment and assignment[other_exam][1] == slot:
```

```
        return False
```

```
# Precedence constraints: "before" must be scheduled earlier than "after"
```

```
for before, after in self.precedence:
```

```
    if exam == after and before in assignment:
```

```
        if self.timeslots.index(assignment[before][1]) >= self.timeslots.index(slot):
```

```
            return False
```

```
    if exam == before and after in assignment:
```

```
        if self.timeslots.index(slot) >= self.timeslots.index(assignment[after][1]):
```

```
            return False
```

```
return True
```

```
def forward_check(csp: ExamCSP, assignment, unassigned_domains, exam, value):
```

```
    """
```

After assigning `exam = value`, prune inconsistent values from the domains of neighboring (still-unassigned) exams. Returns the set of removed (other_exam, removed_value) pairs for backtracking restoration, or None if any domain becomes empty (dead end detected early).

```
    """
```

```
    removed = []
```

```
    related = set(csp.exams_of_same_student(exam))
```

```
    for other in list(unassigned_domains.keys()):
```

```
        if other == exam:
```

```
            continue
```

```
        new_domain = []
```

```

for candidate in unassigned_domains[other]:
    hypothetical = dict(assignment)
    hypothetical[exam] = value
    hypothetical[other] = candidate
    if csp.consistent(other, candidate, {k: v for k, v in hypothetical.items() if k != other}):
        new_domain.append(candidate)
    else:
        removed.append((other, candidate))
if not new_domain:
    return None # dead end -- some future exam has no legal value left
unassigned_domains[other] = new_domain
return removed

```

```

def backtracking_search(csp: ExamCSP, use_forward_checking=True):

```

```

    """

```

```

    Backtracking search over exam assignments. With forward checking enabled,
    it proactively prunes domains of unassigned exams after each assignment,
    detecting dead ends far earlier than plain backtracking.

```

```

    """

```

```

    assignment: Dict[str, Tuple[str, str]] = {}
    domains = {exam: csp.domain(exam) for exam in csp.exams}
    nodes_explored = [0]

```

```

def select_unassigned(domains_state):

```

```

    # Minimum Remaining Values (MRV) heuristic: pick the most constrained exam
    remaining = [e for e in csp.exams if e not in assignment]
    return min(remaining, key=lambda e: len(domains_state[e]))

```

```

def backtrack(domains_state) -> bool:

```

```

    if len(assignment) == len(csp.exams):

```

```
    return True
```

```
    exam = select_unassigned(domains_state)
```

```
    for value in list(domains_state[exam]):
```

```
        nodes_explored[0] += 1
```

```
        if csp.consistent(exam, value, assignment):
```

```
            assignment[exam] = value
```

```
        if use_forward_checking:
```

```
            domains_copy = {k: list(v) for k, v in domains_state.items()}
```

```
            removed = forward_check(csp, assignment, domains_copy, exam, value)
```

```
            if removed is not None:
```

```
                if backtrack(domains_copy):
```

```
                    return True
```

```
        else:
```

```
            if backtrack(domains_state):
```

```
                return True
```

```
    del assignment[exam]
```

```
    return False
```

```
success = backtrack(domains)
```

```
return (assignment if success else None), nodes_explored[0]
```

```
def demo_scenario_4():
```

```
    print("=" * 70)
```

```
    print("SCENARIO 4: University Exam Timetabling (CSP)")
```

```
    print("=" * 70)
```

```

exams = ["Math101", "CS101", "Physics201", "CS102", "Bio150", "Chem110"]
halls = ["HallA", "HallB"]
timeslots = ["Mon-9am", "Mon-2pm", "Tue-9am", "Tue-2pm"]

student_exams = {
    "student1": ["Math101", "CS101", "Physics201"],
    "student2": ["CS101", "CS102", "Bio150"],
    "student3": ["Physics201", "Chem110", "Bio150"],
    "student4": ["Math101", "Chem110"],
}

precedence = [("CS101", "CS102")] # CS101 must happen before CS102
faculty_unavailable = {"Physics201": {"Mon-9am"}} # professor unavailable then

csp = ExamCSP(exams, halls, timeslots, student_exams, precedence, faculty_unavailable)

result_fc, nodes_fc = backtracking_search(csp, use_forward_checking=True)
result_plain, nodes_plain = backtracking_search(csp, use_forward_checking=False)

print("Backtracking + Forward Checking result:")
if result_fc:
    for exam, (hall, slot) in sorted(result_fc.items()):
        print(f" {exam:<12} -> {hall}, {slot}")
    print(f" Nodes explored: {nodes_fc}")

print(f"\nPlain Backtracking nodes explored: {nodes_plain}")
print(f"Forward checking reduced search effort by "
      f"{max(0, nodes_plain - nodes_fc)} node visits.")
print()

# =====

```

```
# SCENARIO 5: REAL-TIME STRATEGY GAME AI
```

```
# Minimax with Alpha-Beta Pruning (demonstrated on Tic-Tac-Toe,
```

```
# generalizable to any turn-based adversarial game)
```

```
# =====
```

```
class TicTacToe:
```

```
    """A minimal but complete adversarial game used to demonstrate Minimax /  
    Alpha-Beta on a real, playable game tree."""
```

```
    def __init__(self):
```

```
        self.board = [" "] * 9 # 3x3 board, flattened
```

```
    def available_moves(self, board):
```

```
        return [i for i, c in enumerate(board) if c == " "]
```

```
    def winner(self, board):
```

```
        lines = [(0, 1, 2), (3, 4, 5), (6, 7, 8),  
                 (0, 3, 6), (1, 4, 7), (2, 5, 8),  
                 (0, 4, 8), (2, 4, 6)]
```

```
        for a, b, c in lines:
```

```
            if board[a] != " " and board[a] == board[b] == board[c]:
```

```
                return board[a]
```

```
        if " " not in board:
```

```
            return "draw"
```

```
        return None
```

```
    def evaluate(self, board):
```

```
        """
```

```
        Evaluation function: +10 for AI ('X') win, -10 for opponent ('O') win,  
        0 for draw/ongoing. In a real RTS game this would factor in  
        material advantage, positional control, unit health, resources, etc.
```

```
"""
```

```
result = self.winner(board)
```

```
if result == "X":
```

```
    return 10
```

```
elif result == "O":
```

```
    return -10
```

```
return 0
```

```
def minimax(game: TicTacToe, board, depth, maximizing, nodes_counter):
```

```
    nodes_counter[0] += 1
```

```
    result = game.winner(board)
```

```
    if result is not None:
```

```
        score = game.evaluate(board)
```

```
        return (score - depth) if score > 0 else (score + depth) if score < 0 else 0
```

```
    if maximizing:
```

```
        best = -math.inf
```

```
        for move in game.available_moves(board):
```

```
            board[move] = "X"
```

```
            best = max(best, minimax(game, board, depth + 1, False, nodes_counter))
```

```
            board[move] = " "
```

```
        return best
```

```
    else:
```

```
        best = math.inf
```

```
        for move in game.available_moves(board):
```

```
            board[move] = "O"
```

```
            best = min(best, minimax(game, board, depth + 1, True, nodes_counter))
```

```
            board[move] = " "
```

```
        return best
```

```
def minimax_alpha_beta(game: TicTacToe, board, depth, alpha, beta, maximizing, nodes_counter):
```

```
    """
```

```
    Alpha-beta pruning: alpha = best value maximizer can guarantee so far,  
    beta = best value minimizer can guarantee so far. Once alpha >= beta,  
    the remaining branches CANNOT affect the final decision and are pruned  
    -- dramatically reducing the effective branching factor without  
    changing the result.
```

```
    """
```

```
    nodes_counter[0] += 1
```

```
    result = game.winner(board)
```

```
    if result is not None:
```

```
        score = game.evaluate(board)
```

```
        return (score - depth) if score > 0 else (score + depth) if score < 0 else 0
```

```
    if maximizing:
```

```
        value = -math.inf
```

```
        for move in game.available_moves(board):
```

```
            board[move] = "X"
```

```
            value = max(value, minimax_alpha_beta(game, board, depth + 1, alpha, beta, False,  
nodes_counter))
```

```
            board[move] = " "
```

```
            alpha = max(alpha, value)
```

```
            if alpha >= beta:
```

```
                break # beta cutoff: prune remaining siblings
```

```
        return value
```

```
    else:
```

```
        value = math.inf
```

```
        for move in game.available_moves(board):
```

```
            board[move] = "O"
```

```
            value = min(value, minimax_alpha_beta(game, board, depth + 1, alpha, beta, True,  
nodes_counter))
```

```
    board[move] = " "  
    beta = min(beta, value)  
    if alpha >= beta:  
        break # alpha cutoff: prune remaining siblings  
return value
```

```
def best_move(game: TicTacToe, board, use_alpha_beta=True):  
    nodes_counter = [0]  
    best_score, chosen = -math.inf, None  
    for move in game.available_moves(board):  
        board[move] = "X"  
        if use_alpha_beta:  
            score = minimax_alpha_beta(game, board, 0, -math.inf, math.inf, False, nodes_counter)  
        else:  
            score = minimax(game, board, 0, False, nodes_counter)  
        board[move] = " "  
        if score > best_score:  
            best_score, chosen = score, move  
    return chosen, best_score, nodes_counter[0]
```

```
def demo_scenario_5():  
    print("=" * 70)  
    print("SCENARIO 5: Strategic Game AI (Minimax & Alpha-Beta Pruning)")  
    print("=" * 70)  
    game = TicTacToe()  
    board = ["X", "O", "X",  
            " ", "O", " ",  
            " ", " ", " "]
```

```
move_plain, score_plain, nodes_plain = best_move(game, board.copy(), use_alpha_beta=False)
move_ab, score_ab, nodes_ab = best_move(game, board.copy(), use_alpha_beta=True)
```

```
print(f"Plain Minimax -> best move: {move_plain}, score: {score_plain}, "
      f"nodes explored: {nodes_plain}")
print(f"Alpha-Beta Pruned -> best move: {move_ab}, score: {score_ab}, "
      f"nodes explored: {nodes_ab}")
reduction = (1 - nodes_ab / nodes_plain) * 100 if nodes_plain else 0
print(f"Alpha-Beta pruning reduced node exploration by {reduction:.1f}% "
      f"while finding the SAME optimal move.")
print()
```

```
# =====
```

```
# MAIN
```

```
# =====
```

```
if __name__ == "__main__":
    demo_scenario_1()
    demo_scenario_2()
    demo_scenario_3()
    demo_scenario_4()
    demo_scenario_5()
    print("=" * 70)
    print("All 5 scenarios completed successfully.")
    print("=" * 70)
```